

CS 374: OPERATING SYSTEMS I

INTRODUCTION TO OPERATING SYSTEMS I

Mon/Wed 12:00 – 1:50 PM (LINC 200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

INSTRUCTOR: SANGHYUN HONG



Who am I?

- **2021 - Now:** Assistant Professor of Computer Science at OSU
- **2021:** Ph.D. from the University of Maryland, College Park
- **2015:** B.S. from Seoul National University, South Korea

What I do?

- **Formal:** I work at the intersection of security, privacy, and machine learning
- **Informal:** I “hack” machine learning, expose security threats and defeat them

What do I teach?

- CS 344: Operating Systems I
- CS 370: Introduction to Security and CS 578 Cyber-security
- CS 499: Machine Learning Security (> AI539 Trustworthy Machine Learning)

Where can you find me?

- **Office:** 2029 Kelley Engineering Center (KEC)
- **Email:** sanghyun.hong (at) oregonstate.edu | Discord Server

TOPICS FOR TODAY

- Course overview
 - Prerequisites
 - Course information (time, location, teams, office hours, ...)
 - Course structure
 - Tips: how to be successful
- Introduction
 - What is an OS?
 - Why do we study OS?
 - Why do we think studying OS difficult?
 - How has OS been developed?
 - What are the functionalities of OS?
 - What are the course topics?

PREREQUISITES

- Courses:
 - CS 261: Data Structures (or similar)
- Skills:
 - Good “problem-solving skills”
 - Some familiarity in C and Bash shell script languages
- Others:
 - PC or a laptop where you can access the OS1 Server for the assignments

COURSE INFORMATION

- Time and location
 - **Time:** 12– 1:50 PM PST (M/W)
 - **Classroom:** #200 LINC or Zoom (**No recordings**)
- Contacts
 - **SH:** sanghyun.hong@oregonstate.edu
 - **TA:** Jose Sanchez Gonzalez (sanchej7@oregonstate.edu)
Wang Bo (wangb3@oregonstate.edu)
 - **Online discussion:** Discord server (see Canvas for the joining link)

COURSE INFORMATION

- Office hours
 - **Location:** on Zoom (see Canvas for the links)
 - **Time:** 22 hrs / week

Time	Mon	Tue	Wed	Thu	Fri
10:00 AM	Jose	Jose	Jose		
10:30 AM					
11:00 AM				Sanghyun	Sanghyun
11:30 AM					
12:00 PM					
12:30 PM					
1:00 PM					
1:30 PM					
2:00 PM		Bo	Jose	Bo	Bo
2:30 PM					
3:00 PM					
3:30 PM					
4:00 PM	Jose				
4:30 PM					
5:00 PM					
5:30 PM					
6:00 PM					

COURSE STRUCTURE

- Tasks
 - 4 Midterm quizzes
 - 5 Programming assignments (reduced from 6 > 5)
 - N+ Extra credit opportunities
- Grading
 - 60%: 5 Programming assignments
 - 40%: 4 Midterm quizzes
 - N%: Extra credit opportunities (on top of the 100% from the above two)

COURSE STRUCTURE

- Grading policy
 - 4 Midterm quizzes
 - **Period:** in every 2-3 weeks, you will have an online quiz (on Canvas)
 - **Method:** You will have 3 times to take each quiz
 - **Timed exam:** 80-120 min
 - 5 Programming assignments
 - **Period:** in every 1-3 weeks, you will have a programming assignment (on Canvas)
 - **Submission penalty:**
 - 0% penalty if you submit the assignment on time
 - 5% penalty for every 24 hours
 - 50% maximum penalty if you submit until the 12nd of June
 - 100% penalty if you miss these deadline without any note
 - N+ Extra credit opportunities
 - **Ad-hoc opportunities:** up to SH

COURSE STRUCTURE

- Tentative schedule

- See: <https://true-lab.ai/courses/OS1/current/syllabus.html>

Schedule

*[Note] This is a *tentative* outline; the lecture contents or the deadlines can change depending on progress.

Date	Topics	Notes	Supplementary Materials
Overview and Motivation			
Mon. 03/30	Intro to Operating Systems	[Slides]	
Wed. 04/01	Preliminaries	[Slides]	[Reading] The Missing Semester of Your CS Education
Part I: OS Fundamentals			
Mon. 04/06	Processes and Threads	[Slides]	[Due] Syllabus Quiz
Wed. 04/08	Context Switching	[Slides]	
Mon. 04/13	Virtual Memory	[Slides]	[Due] Programming Assignment I [Reading] Thrashing
Wed. 04/15	Scheduling (Basics)	[Slides]	[Reading] Process scheduling [Reading] Real-Time Operating Systems (RTOS)

TIPS: HOW TO BE SUCCESSFUL

- Rules:

- **Dont's**

- Do not share your code with others
 - Do not copy and paste someone else's code in yours
 - Do not cheat in online quizzes
 - Do not ask for the solutions on the Internet, *e.g.*, StackOverflow

- **Do's**

- Brainstorm ideas
 - Discuss basic concepts on Discord
 - Help someone else debug if they run into a technical wall

- Tips:

- **No. 1:** Start programming assignment **early**
 - **No. 2:** Come to classes and office hours

TOPICS FOR TODAY

- Course overview
 - Prerequisites
 - Course information (time, location, teams, office hours, ...)
 - Course structure
 - Tips: how to be successful
- Introduction
 - What is an OS?
 - Why do we study OS?
 - Why do we think studying OS difficult?
 - How has OS been developed?
 - What are the functionalities of OS?
 - What are the course topics?

WHAT IS AN OPERATING SYSTEM?

- **Definition**

- Computer software that
lies between hardware and applications

Humans Run Applications



Operating System (OS)

Hardware (CPU, GPU, Mem, ...)



WHY DO WE STUDY OS?

- Many reasons:
 - To graduate / to get a job
 - To make new hardware and use it
 - To make existing OS (*e.g.*, iOS and Linux) better
 - To understand computer systems better
 - To make your software more secure and private (**me...?!)**
 - Just for **fun!**
- **Mine:**
 - OS is an **exciting** field of study
 - OS brings many areas in computer science altogether
 - Data structure, algorithms
 - Programming languages, compilers
 - Computer hardware, architecture

WHY DO WE THINK OS IS DIFFICULT TO STUDY?

- Many reasons:
 - OS requires to understand many computer science areas
 - OS is large (10M+ lines of code; 1000+ man-years of work)
 - OS is complex (many different behaviors; hardware supports; goals)
 - OS is poorly understood (only few completely understood Linux internals)
 - ... **A false sense of difficulty!**

My Guess: We Are Not Familiar to How Computers Think!

WHY DO WE THINK OS IS DIFFICULT TO STUDY?

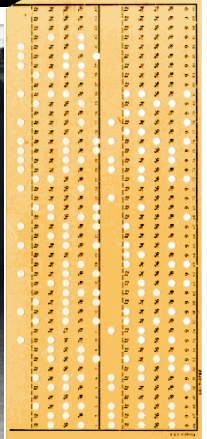
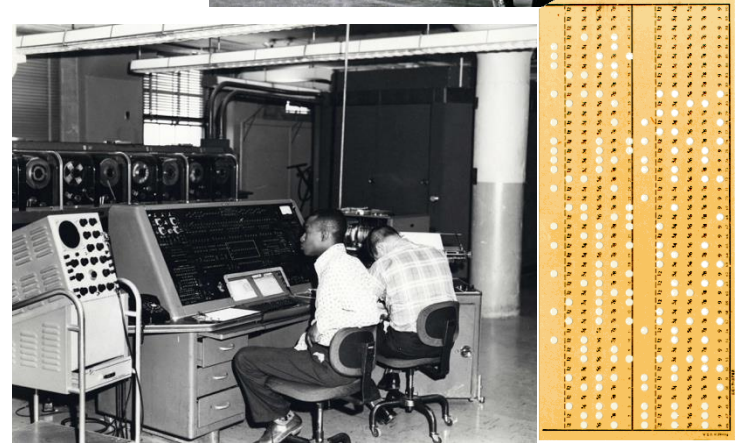
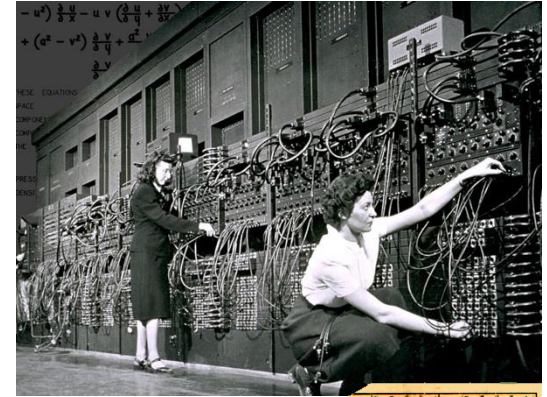
- Study tips for CS 344:
 - Focus on **problems**: OS is a history of “problem-solving”
 - Memorize **definitions**: we name things with meanings
 - Understand **solutions** and **their limits**
 - Do hands-on exercises: write your own *small* programs

HOW HAS OS BEEN EVOLVED?

- Why do we care about this history?
 - Defining OS *precisely* is difficult
 - Motivation:
 - To run computer programs
 - OS has been evolved to solve “a set of problems”
 - Problems: new hardware, faster execution, multi-program supports, etc...
- Three Phases of OS History
 - Phase I: early 50s – mid 60s
 - Phase II: mid 60s – mid 90s
 - Phase III: mid 90s – Present
 - Phase IV: It's your job!

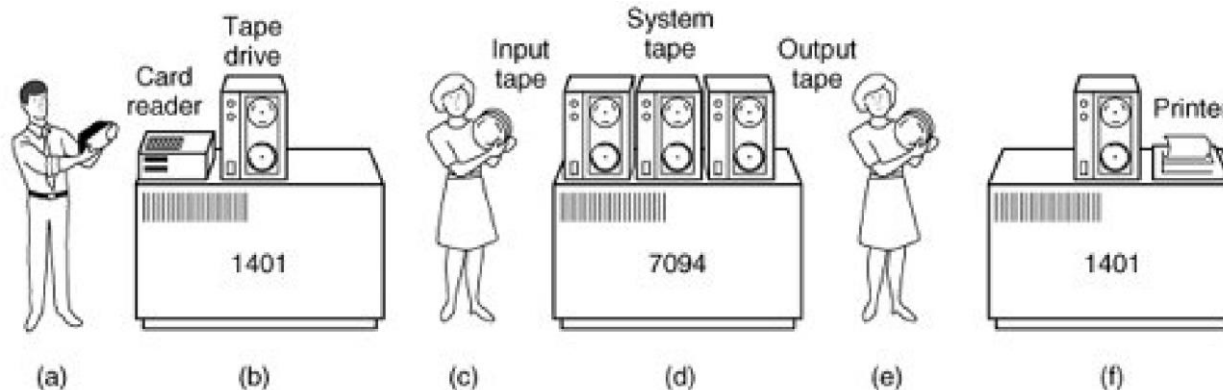
PHASE I: EARLY 50s – MID 60s

- Motivation
 - Hardware was expensive; humans are cheap
- Design objectives
 - Make efficient use of the hardware
 - Increase CPU utilization (no idle time)
- Phase I-I: Human Operator as OS
 - OS (Human) is a shared subroutine (function)
 - Only one user can run a job at a time
 - Job to job transition is slow (human...)



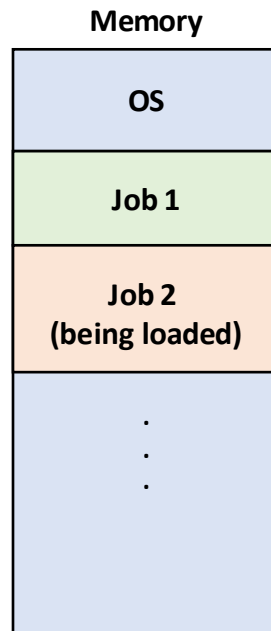
PHASE I: EARLY 50s – MID 60s

- Phase I-II: *Simple Batch Monitor*
 - OS loaded: run user jobs and take dumps
 - IBM machines
 - IBM 1401 (I/O Machine) reads cards in batch onto tape
 - IBM 7904 (Main Machine) computes and dumps results back to the tape
 - I/O Machine prints outputs from the tape



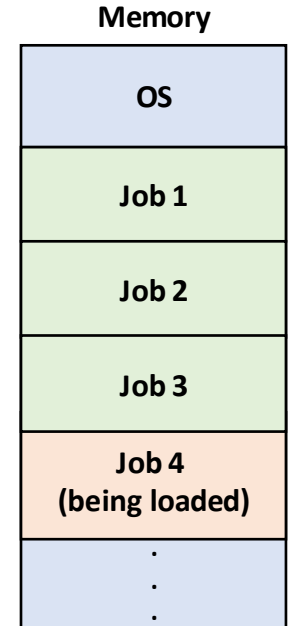
PHASE I: EARLY 50s – MID 60s

- Problem of Simple Batch Monitors
 - CPU in idle when the machines do I/O
 - CPU stops if a job in a tape has errors
- Phase I-III: **Batch Monitor**
 - **Solution:** let's make CPUs run while the machine is on I/O (overlap!)
 - **Interrupt (asynchronous)** mechanism:
 - Machine first reads a tape
 - Machine runs jobs in the tape and starts reading another tape
 - If the tape reading is done, the machine sets “ready” flag.
 - Do iteratively...



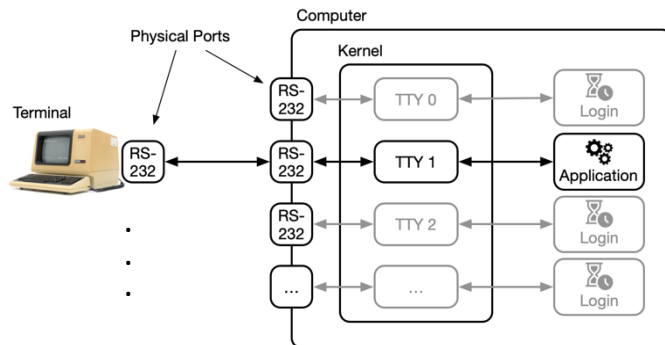
PHASE I: EARLY 50s – MID 60s

- Problem of Batch Monitors
 - Run only a single job at a time
 - What if a job does some I/O? All other jobs need to wait...
- Phase I-IV: *Multi-programmed* Batch Monitor
 - **Solution:**
 - Let's make an OS run/manages multiple jobs
 - *e.g.*, load a new job to CPUs until the I/O is done
 - OS became a focus of study
 - How can we *protect* multiple jobs in mem?
 - How can we manage multiple jobs in mem with a *smart* way?
 - How can we *write* programs that maximize the CPU usage?
 - Memory protection; relocation; concurrency...



PHASE II: MID 60s – MID 90s

- Motivation
 - Hardware became cheap; humans are expensive
- Design objectives
 - Make efficient use of human's time
 - Reduce the idle time of humans
- Phase II-I: Interactive **Time-sharing** OS
 - **Solution:** Give terminals (cheap) to users
 - **Problems:**
 - CPU Time should be sliced ...
 - Not all data can be accessed by everyone ...
 - New metrics for evaluating OS ...



PHASE III: MID 90s – PRESENT

- Motivation
 - Computers became connected to each other
- Design objectives
 - Offer connected multimedia services for users
- Phase III-I: OS Built with **Connectivity**
 - Internet *protocols* added to PC OS
 - Internet (network) programming became important (Web, Python, ...)
 - Multi-tasking became much more important

PHASE III: MID 90s – PRESENT (CONT'D)

- Phase III-II: **Complex** PC OS
 - Computers became extremely cheap
 - Computers were equipped with sophisticated H/W architecture
 - Modern OS should be complex too to support H/W...
- Phase III-III: OS with **Multimedia** Support
 - Increasing demands of computer and network resources
 - Human perception became the center of universe
 - QoS (Quality of Services)
 - RTOS (Real-time OS)
 - Home appliances and computers became merged (IoT devices...)

WHAT DOES OS DO?

- Functionalities of Modern OS
 - Manage resources
 - Provide abstractions
 - Offer standard interfaces

Humans Run Applications



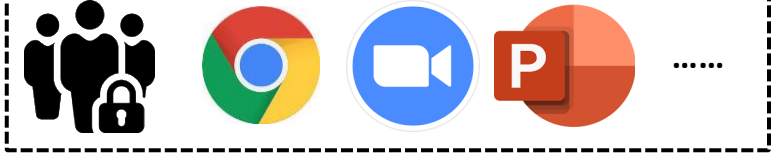
Hardware (CPU, GPU, Mem, ...)



WHAT DOES OS DO?

- Functionalities of Modern OS
 - **Manage resources**
 - Provide abstractions
 - Offer standard interfaces

Humans Run Applications



Manage CPU, Memory, Networking, Storage...

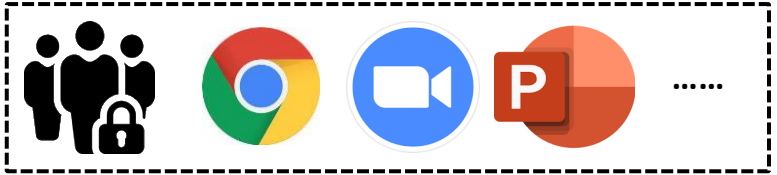
Hardware (CPU, GPU, Mem, ...)



WHAT DO OS DO?

- Functionalities of Modern OS
 - Manage resources
 - **Provide abstractions**
 - Offer standard interfaces

Humans Run Applications



Manage CPU, Memory, Networking, Storage...

H/W Abstractions

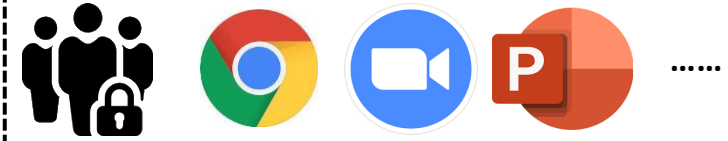
Hardware (CPU, GPU, Mem, ...)



WHAT DO OS DO?

- Functionalities of Modern OS
 - Manage resources
 - Provide abstractions
 - **Offer standard interfaces**

Humans Run Applications



Standard Interfaces (Libraries)

Manage CPU, Memory, Networking, Storage...

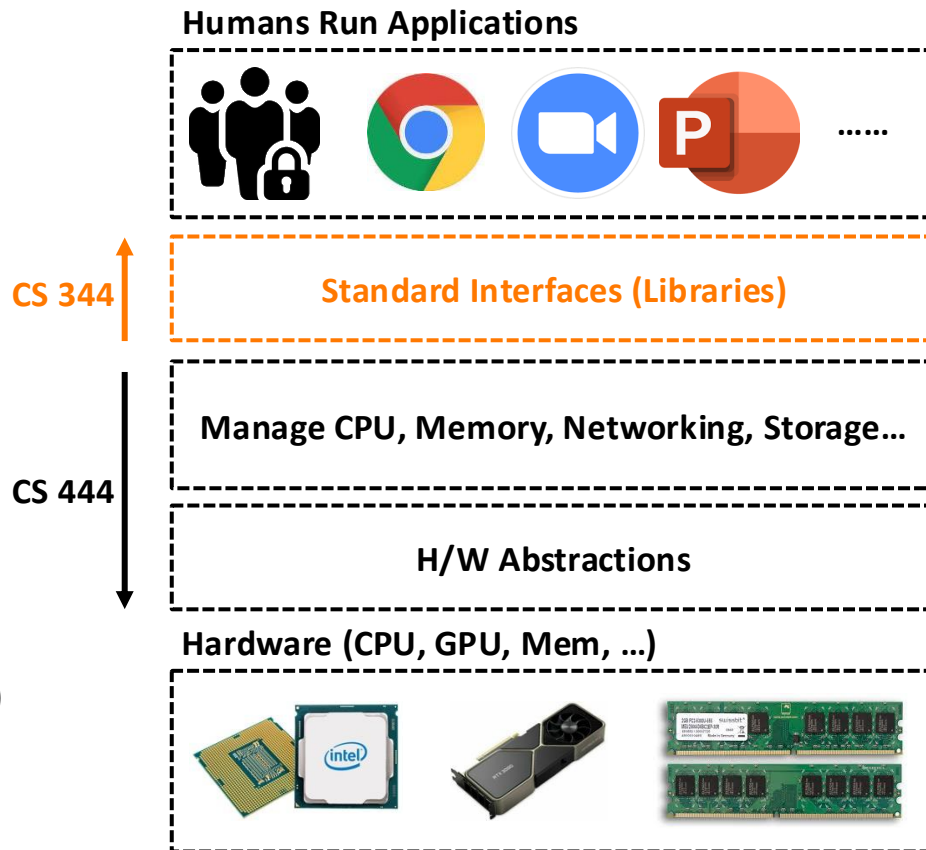
H/W Abstractions

Hardware (CPU, GPU, Mem, ...)



WHAT ARE THE COURSE TOPICS?

- Functionalities of Modern OS
 - Manage resources
 - Provide abstractions
 - Offer standard interfaces
- What do we learn in CS 344?
 - Standard interfaces
 - Shell (Bash), Terminal
 - C language
 - Files and directories
 - Processes (and threads)
 - Inter-process communication (IPC)
 - Networking
 - (Secure OS) Rust language



TOPICS FOR TODAY

- Course overview
 - Prerequisites
 - Course information (time, location, teams, office hours, ...)
 - Course structure
 - Tips: how to be successful
- Introduction
 - What is an OS?
 - Why do we study OS?
 - Why do we think studying OS difficult?
 - How has OS been developed?
 - What are the functionalities of OS?
 - What are the course topics?

Thank You!

Mon/Wed 12:00 – 1:50 PM (LINC 200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI